

Mock Test 2

Started: Aug 5 at 9:25am

Quiz Instructions



Question 1 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI pipeline generates thousands of test cases across many PRs. The standard API would be very expensive. Which Claude API feature should you use?



Clustering API to group similar test requests



Cache API to reuse test generation logic



Message Batches API for 50% cost savings on non-interactive requests



Streaming API to reduce token usage



Question 2 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

You want code review feedback to be specific and testable. Instead of “improve error handling,” what should you specify?



“Flag when exceptions are caught but not logged or re-raised” with code examples



“Review error handling quality”



“Ensure code is robust against failures”



“Check for good error handling practices”



Question 3 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI code review uses a JSON schema to return findings with fields: file, line, severity, issue_type, and description. Should you include an “other” field for unexpected issues?



No, add more categories instead of a catch-all



Yes, include an optional “other” field with detail string for issues that don’t fit standard categories



No, strictly defined categories ensure predictable output



Yes, but make it required to ensure all issues are captured



Question 4 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI system includes Claude Code’s findings in pull request comments. Different team members see duplicate or contradictory comments in the same PR. Why might independent review instances be better?



Independent instances require less configuration than a single instance



Independent instances run in parallel, completing reviews faster



Independent instances eliminate bias completely



Independent instances catch issues the other misses; fresh perspective reduces blind spots



Question 5 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

You want to provide a few-shot example for your CI review prompt showing how to handle a security issue. What should you include?

☐

Multiple examples with different severity levels

☐

Just the issue code without context

☐

A generic description of security issues and best practices

☒

Code snippet with the issue, the specific problem to flag, and the expected feedback format



Question 6 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI pipeline runs Claude Code with `--output-format json` to integrate results into your dashboard. What flag should you also use to ensure output matches your expected schema?

☒

`--validate-schema` to check output validity

☐

`--strict-json` for enforcement

☐

`--schema-compliance` to verify schema adherence

☐

`--json-schema` to specify the exact output structure expected



Question 7 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI CLAUDE.md file specifies testing standards and review criteria. When Claude Code runs in the CI pipeline, should it have access to this configuration?



Only if critical tests are involved; not all tests need team standards



Yes, CI-invoked Claude Code should access CLAUDE.md so tests and reviews match your team's standards



No, CI should use independent standards separate from development standards



Use a separate CI-specific CLAUDE.md for automation



Question 8 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI code review system is designed to use multiple independent reviewer agents, each focusing on specific domains (performance, security, style). How should each agent get its analysis tools restricted?



Use allowedTools in the agent definition to give each only domain-relevant tools



Give all agents the same full tool set for consistency



Configure tool access in CLAUDE.md per agent



Trust each agent to use tools responsibly within their domain



Question 9 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI test generation uses Claude Code with MCP tools. An MCP tool returns an error with isRetryable: true. How should Claude Code respond?



Retry the tool call since the error is transient



Treat all errors equally regardless of isRetryable



Log the error and continue



Skip to the next tool immediately



Question 10 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

You have a custom MCP tool for database schema analysis. The tool has three operations: `list_tables`, `check_integrity`, and `analyze_performance`. How should you design this?



One tool per database type (PostgreSQL, MySQL, etc.)



One tool that always runs all operations and returns combined results



Three separate tools (one per operation) for clear separation



One tool with clear description and operation parameter letting Claude Code choose which to run



Question 11 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

When designing a JSON schema for extracted data, what fields should always be included alongside the extracted data?



Full extraction logs for audit trails



Extracted fields plus `confidence_score`, `extraction_source`, and `extraction_timestamp` for traceability



Only the extracted fields needed by downstream systems



Extracted fields plus error_handling_metadata for debugging



Question 12 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system encounters documents with conflicting or ambiguous information (e.g., a product name is listed differently in two places). What should the extraction schema require?



Return an error and ask for human clarification



Extract all versions found and include a detected_conflict boolean with explanation of the conflict



Extract the most common version



Extract whichever version appears first



Question 13 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer says they have been going back and forth on a return for days and want to speak to someone who can actually help. The agent has confirmed the return is straightforward and eligible for immediate processing. What should the agent do?



Acknowledge frustration, explain this is resolvable now, and offer to complete it or escalate.



Ask what specifically has not worked in previous attempts before deciding whether to escalate or resolve automatically.



Call escalate_to_human immediately to honor the customer's request.



Process the refund to resolve the issue, then tell the customer it is complete.



Question 14 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your **CI/CD pipeline**. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI process uses Claude Code with the **-p flag** in a GitHub Actions workflow.



Non-interactive mode, allowing Claude Code to run without requiring approvals or human interaction



Preview mode, showing potential changes without applying them



Parallel execution of multiple Claude Code instances simultaneously



Performance profiling to measure execution time



Question 15 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer says they are frustrated, have explained the issue twice, and want to talk to a real person now. The agent has not yet called any tools to investigate the account. What should the agent do?



Immediately call `escalate_to_human` with the conversation history.



Acknowledge the frustration and ask one targeted question before escalating.



Briefly explain what the agent can help with and offer to resolve the issue quickly, escalating only if the customer repeats the request.



First call `get_customer` and `lookup_order` to gather account context, then escalate.



Question 16 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, `CLAUDE.md` configurations, and understand

when to use plan mode vs direct execution.

You're implementing a hub-and-spoke coordinator-subagent pattern for code review. The coordinator receives pull request feedback from multiple review subagents. What's the key architectural benefit of this pattern? The pattern eliminates the need for tool_choice configuration



The coordinator can control all subagents synchronously, ensuring sequential execution



The pattern eliminates the need for tool_choice configuration



Subagents share the main context, reducing memory usage



Each subagent operates with isolated context, preventing context pollution and allowing parallel specialization



Question 17 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your main support agent is configured with allowedTools: ["get_customer", "lookup_order", "process_refund", "escalate_to_human"]. You're concerned about scope creep where the agent might attempt operations outside its domain. What's the implication of this scoped tool access?



The agent's system prompt is restricted, but it can request tools outside this list



The agent will attempt all possible operations but these four will have priority



The agent can only call these four tools; any attempt to call tools outside this list will fail at runtime



Tool scoping is only a recommendation; the agent will ignore it if it seems necessary



Question 18 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

You have a tool called get_customer that's critical for retrieving customer details. Your tool description

currently says: 'Retrieves customer information.' A team member says this is sufficient. Why might this description be inadequate for reliable agent behavior?



The description is fine; agents don't need detailed tool descriptions to use tools correctly



The description is too vague. It doesn't specify what parameters are required, what output format to expect, what edge cases exist (e.g., customer not found), or when to use this tool vs. other alternatives



Tool descriptions should only be one sentence for brevity



Detailed descriptions slow down the model's reasoning process



Question 19 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

You're designing the `process_refund` tool. Your backend returns different failure modes: 'Insufficient funds in account', 'Account locked pending fraud review', and 'Refund window expired'. How should you structure the tool's error response?



Throw exceptions and let the SDK handle error translation automatically



Return success even on failure and embed error codes in the result for the agent to parse



Use `isError: true` with `errorCategory` (`transient`, `non_retryable`, etc.), `isRetryable` (boolean), and a human-readable details message so the agent can reason about next steps



Return all error messages as plain text strings without categorization



Question 20 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your MCP server defines tools at the user level (`~/claude.json`). Your team wants different projects to

have different sets of tools. For example, one project uses `process_refund`, another uses `process_replacement_order`. How should you structure MCP server scoping?



Define project-level `.mcp.json` files so each project can have scoped tool access independent of others



Define all tools in `~/.claude.json` and let each project's agent choose which ones to use via `tool_choice`



Store tool definitions in a database and fetch them at runtime based on the project



Require each project to copy the full `~/.claude.json` and edit it locally



Question 21 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

You want the agent to dynamically decide which tool to call based on the situation. You're choosing between `tool_choice: 'auto'` and `tool_choice: 'any'`. What's the practical difference?



`auto` and `any` are equivalent; both let the model choose any available tool



`any` is more restrictive and prevents the model from calling certain tools



`auto` prevents the model from using tools at all



`auto` uses model heuristics to decide whether and when to call tools; `any` forces the model to always call a tool in every response, even if it might not need one



Question 22 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your Claude Code project has a `CLAUDE.md` file with custom system instructions and rules for the support agent. You're adding a new project dependency (a special tool for handling warranty claims). Should you add the warranty tool definition to the root `CLAUDE.md` or use the `@import` syntax?

☐ Always add everything to the root CLAUDE.md to keep configuration centralized

☒ Use @import to modularize the CLAUDE.md so different features (refunds, returns, warranty claims) are defined in separate files and imported as needed

☐ Create a separate CLAUDE.md in each subdirectory and ignore the root file

☐ Configuration should never be split across files; it violates best practices



Question 23 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're setting up Claude Code to work with your team's MCP resources. Your catalog contains hundreds of documentation pages. How should you structure this for efficient usage?

☐ Create a custom tool that searches the documentation using full-text search

☐ Load all documentation into the initial prompt context

☐ Store documentation paths in CLAUDE.md for reference

☒ Use MCP resources to create a catalog index that Claude Code can query without reading every page



Question 24 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You want to restrict a subagent to use only specific tools: Grep, Glob, and Read. How do you configure this using AgentDefinition?

☐ Use tool_choice to force only those tools



Set allowedTools: ["Grep", "Glob", "Read"] in the AgentDefinition



Create a skill with allowed-tools in the frontmatter



Add tool restrictions to CLAUDE.md for the subagent



Question 25 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're designing a multi-step code generation workflow. Step 1 generates a module skeleton, Step 2 adds implementations, and Step 3 creates tests. All steps depend on the previous output. Should you use prompt chaining or dynamic task decomposition?



Dynamic task decomposition, which adapts based on intermediate results



Prompt chaining only if you can predict all possible outcomes in Step 1



Prompt chaining, since each step follows predictably from the previous one



Both equally; the choice doesn't matter for structured workflows



Question 26 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're spawning a code analysis subagent using the Task tool. You want to limit it to reading files and searching patterns without modifying the codebase. What should you specify in the Task configuration?



A tool_choice configuration forbidding write operations



A restrictive system prompt instructing the subagent not to modify files



allowedTools configuration restricting access to Grep, Glob, and Read only



A CLAUDE.md file specifically for the subagent



Question 27 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're designing a code generation prompt that should produce Python functions. You want Claude Code to always return valid Python with no syntax errors. What's the most effective approach?



Include several correct Python function examples in the prompt



Ask Claude Code to validate Python syntax before responding



Use a custom linter tool to check output



Use tool_use with a JSON schema specifying function code and expected behavior



Question 28 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're using Claude Code to extract structured data from code comments. The extraction should be consistent and handle edge cases. What approach ensures reliability?



Use a verbose prompt explaining the task thoroughly



Request Claude Code to be extra careful with extraction



Retry extraction multiple times and pick the best result



Provide few-shot examples showing expected extraction format for edge cases



Question 29 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're building a code review prompt that checks for security issues. The criteria should be explicit rather than vague. Which approach is better?

☐

"Rate security overall" rather than listing specific checks

☒

"Flag when code accepts user input without validation" rather than "Check for security issues"

☐

"Use your security knowledge to find issues" rather than defined criteria

☐

"Identify problematic code" rather than specific security patterns



Question 30 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're designing a schema for Claude Code to return code review comments. The schema has optional fields like "suggestion" and "severity". Should you include an "other" field for unexpected issues?

☒

No, strictly defined enum fields ensure predictable output

☐

No, add more specific fields instead of a catch-all

☐

Yes, but make it required to capture all issues

☐

Yes, include "other" with detail field to handle unforeseen issues without breaking schema



Question 31 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're processing thousands of code review requests in batch. The Message Batches API costs 50% less but has constraints. What's a key limitation to be aware of?



No multi-turn tool calling; each request is independent



Requires pre-payment for all requests



Cannot be used for code review tasks



Longer latency requiring 24-hour completion window



Question 32 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're implementing a code quality checker for a large codebase. To avoid noise, you want to perform a multi-pass review: first per-file analysis, then cross-file integration checks. Why is this better than a single comprehensive review?



Single review is faster; multi-pass is unnecessary overhead



Multi-pass enables focused analysis per scope, reducing hallucinations and improving precision



Multi-pass allows parallelization for speed improvement



Single review risks missing issues; multi-pass provides redundancy



Question 33 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're training independent review instances for a distributed code review system. Should each instance self-review its own findings for accuracy?



Self-review and independent review are equivalent



Use self-review only for critical issues



No; independent review instances are more effective than self-review for catching errors



Yes; each instance should validate its own findings



Question 34 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're building a code review system and getting too many false positives flagging suspicious patterns that are actually correct. What's the best approach?



Tell Claude Code to reduce false positives in the system prompt



Define specific review criteria for each category and temporarily disable high false-positive categories



Increase the confidence threshold to filter lower-confidence findings



Use a veto process where engineers manually approve each finding



Question 35 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team's code generation process is producing duplicated variable names in some cases. What's the best approach to improve consistency?



Use a post-processing linter to rename duplicates



Add naming convention rules to CLAUDE.md



Tell Claude Code to check for duplicates before completing



Provide test cases showing expected variable naming and how duplicates should be handled



Question 36 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI pipeline runs automated code review on every PR. For a codebase with hundreds of files changed, how should you structure Claude Code to avoid overwhelming developers with feedback?



Run per-file analysis first, then cross-file integration checks in separate passes



Analyze all files together in one comprehensive review for consistency



Review only files that changed significantly to reduce noise



Run multiple independent reviewers and aggregate findings



Question 37 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your code review system is flagging legitimate performance idioms as inefficient code. The false positives frustrate developers. What's the best approach to reduce this noise?



Require manual approval before any performance feedback is posted



Temporarily disable the performance check category while you refine the criteria with concrete examples



Tell Claude Code to be less aggressive in the review prompt



Filter out low-confidence flags to reduce false positives



Question 38 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your team is extracting structured data from 50,000 legacy contracts under a two week deadline. A representative sample shows that many documents pass on the first attempt but failures typically need prompt refinements based on failure modes. Which batch processing strategy is the most cost effective while still meeting the deadline?

☐

Submit all 50,000 documents via batch API, then rebatch failures successively while refining prompts between rounds.

☐

Process a representative sample via real time API to identify failure patterns and refine prompts, then batch process all 50,000 with the optimized prompts.

☐

Use the real time API for all 50,000 documents because the batch API window is too risky.

☐

Split the documents into many sequential batches and refine prompts between batches.



Question 39 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema. Downstream systems reject malformed or non conformant JSON. What approach provides the most reliable schema compliance?

☐

Define a tool with the target schema as input parameters and have the model call it with the extracted data

☐

Pre fill the response with an opening brace and parse the completed text as JSON.

☐

Include detailed JSON formatting instructions and the target schema in the prompt.

☐

Append strict JSON only instructions and use retry logic if parsing fails.



Question 40 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

The extraction pipeline receives multiple document types, each with its own extraction tool and schema. With `tool_choice` set to `auto`, the model sometimes returns conversational text instead of calling a tool. You need guaranteed structured output without knowing the document type in advance. What is the most effective approach?



Add a preliminary classification call, then make a second call with `tool_choice` forced to the identified extraction tool.



Set `tool_choice` to `any` with all extraction tools defined.



Consolidate all document types into a single unified schema extraction tool and force that tool.



Keep `tool_choice` as `auto` and add system prompt instructions requiring tool use.



Question 41 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline processes restaurant menus and must output structured JSON with fields for item names, descriptions, prices, and dietary tags. Menus use inconsistent formatting. What is the most reliable approach?



Define a strict output schema and include format normalization rules in the prompt.



Request multiple extraction attempts and select the most common format.



Extract data as is and normalize formats in post processing code afterward.



Use separate extraction calls for each field.



Question 42 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON)

schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system retries when validation fails, appending the validation error to the prompt each time. For which failure pattern would additional retries be least effective?



The model outputs a nested object where the schema requires a flat array.



The model outputs et al. for co authors when the full list exists only in an external document not present in the input.



The model outputs ISO 8601 datetimes where the schema requires only dates.



Model outputs locale-formatted strings where the schema requires integers



Question 43 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline processes invoices and in many cases the sum of extracted line items does not match the extracted grand total. Downstream systems reject mismatched totals. What is the most effective approach to improve reliability?



Add a `calculated_total` field alongside the `stated_total` and flag records for human review when they differ.



Automatically adjust line item amounts proportionally when totals do not match



Add few shot examples encouraging the model to produce mathematically consistent extractions.



Extract line items and totals independently, then use a separate validation model to reconcile discrepancies.



Question 44 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your PR test generation system is suggesting tests identical to existing tests in the file. How can you prevent this?



Include existing test file content in the context so Claude Code sees what's already tested



Ask Claude Code to suggest novel test cases not seen before



Generate tests in a separate CI job to avoid context pollution



Use a database to track tested scenarios and filter duplicates



Question 45 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

After deployment, you find that some extractions contain semantic errors that pass schema validation, and reviewers can check only a subset of outputs. Which approach most effectively allocates reviewer attention?



Prioritize review of all extractions where required fields are empty or marked not found.



Have the model output field level confidence scores and calibrate review thresholds using a labeled validation set.



Randomly sample 20 percent of extractions for review.



Review all extractions from documents with formatting anomalies.



Question 46 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

After implementing tool use with strict schema definitions, syntax errors disappear, but some extractions still have valid JSON with empty strings or null values for required fields even though the source documents contain that information in varied formats. What is the most effective way to address these failures?



Add few shot examples showing extractions from documents with varied structures and how to locate the required information.



Implement retry logic that re sends requests when validation detects empty required fields.



Build a regex based post processing layer that scans source documents for likely citation and methodology patterns.



Modify the schema to make those fields optional and flag incomplete records for manual review.



Question 47 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Documents arrive continuously during business hours. You want to use the batch API to reduce costs, and your SLA requires results within 30 hours of arrival. Which batching strategy is most appropriate?



Submit batches every 4 hours containing documents from that window.



Use the real time API for all documents instead of batch processing.



Submit a single end of day batch containing all documents from that day.



Submit batches every 6 hours containing documents from that window.



Question 48 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Monitoring shows many extractions fail Pydantic validation with specific errors such as expected float for quantity but got a range value like 2 to 3. Retrying without modification produces identical failures. What is the most effective recovery?



Send a follow up request including the validation error and ask the model to correct its output.



Use a larger model tier to reprocess failed documents.



Pre process source documents to standardize problematic formats before extraction.



Set temperature to zero to ensure consistent formatting.



Question 49 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

The system needs to extract candidate information from resumes into a predefined JSON schema, and missing fields or wrong types will break downstream validation. What is the most reliable approach?



Define a tool with an input schema matching the required structure and extract the data from the tool call response.



Make two API calls, first extracting text and then converting it to JSON.



Use detailed JSON instructions and a template example in the prompt.



Parse the model's text response with regex and retry malformed outputs.



Question 50 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system parses e-commerce product descriptions to extract fields such as dimensions, weight, and materials. The model inconsistently extracts the materials field and sometimes omits it even when the information is present. What is the most effective way to improve consistency?



Make the materials field required instead of optional.



Switch to a more capable model tier.



Add few shot examples showing complete input output pairs with standardized material descriptions.



Set temperature to zero for deterministic output.



Question 51 15 pts

Scenario: Structured Data Extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

When extraction returns a null value for a required field, how should the system respond?

☐

Return an error and retry extraction with a modified prompt

☐

Accept null values and pass them to downstream systems

☐

Include the null and document why extraction failed (field not found, ambiguous, etc.)

☐

Generate a placeholder value



Question 52 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

After investigating a billing dispute over many turns, you have identified duplicate charges caused by payment gateway retry logic. The required refund exceeds your authorization limit, and the human agent will not have access to the transcript. What context should you pass when escalating?

☐

The complete conversation transcript with all tool results.

☐

The customer's original complaint plus excerpts of tool results showing duplicate transactions.

☐

A structured summary with customer ID, root cause, refund amount, and recommended action.

☐

Your diagnosis and the refund amount only.



Question 53 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while

knowing when to escalate.

When the agent calls `lookup_order` and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call `process_refund` or `escalate_to_human` next?



The order details are added to the conversation and the model reasons about which action to take.



The agent executes the remaining steps in a tool sequence planned at the start of the request.



The orchestration layer automatically routes to the next tool based on the order's status field.



The agent follows a pre configured decision tree mapping order attributes to specific tool calls.



Question 54 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

You are implementing escalation logic for when the agent should call `escalate_to_human`. Which approach will most reliably identify cases that genuinely require human intervention?



Escalate after three consecutive tool calls fail to resolve the issue.



Use sentiment analysis and escalate when frustration exceeds a configured threshold.



Escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.



Build a rules engine mapping issue types, customer segments, and product categories to escalation decisions.



Question 55 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer raises three separate issues during one session, and the conversation is approaching

context limits. What strategy best maintains the agent's ability to address all issues throughout the session?



Rely on MCP tools to re fetch relevant information on demand when the customer references earlier issues.



Extract and persist structured issue data such as order IDs, amounts, and statuses into a separate context layer.



Summarize earlier turns into a narrative description and preserve full history only for the active issue.



Implement a sliding window that retains only the most recent turns.



Question 56 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer returns four hours after their initial session. Resuming with stale tool results causes the agent to reference outdated information even after fresh tool calls provide newer facts. What approach most reliably handles returning customers?



Resume with full history and automatically re call all previously used tools at session start.



Start a new session, inject a structured summary of the previous interaction, then make fresh tool calls before engaging.



Resume with full history and add a system prompt telling the agent to always prefer the most recent tool results.



Resume with full history but filter out previous `tool_result` messages, keeping only the human and assistant turns.



Question 57 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

You're implementing the agentic loop and encounter a situation where `process_refund` returns an

error: "Refund failed: Account is locked pending fraud review". This is a retryable scenario in 6 hours. How should you communicate this to the agent?



Return a success result and let the agent figure out what happened



Return the error as-is from the backend without any transformation or categorization



Immediately call `escalate_to_human` without telling the agent about the error details



Return an error result with `isError: true`, `errorCategory: 'transient'`, `isRetryable: true`, and a human-readable message explaining the fraud hold and suggesting escalation



Question 58 15 pts

Scenario: Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your `process_refund` tool returns technical errors that are transient and business errors that are permanent, but both currently return only plain text. What is the most effective way to reduce wasted retries while improving customer facing response quality?



Add few shot examples showing how to distinguish retrieable from non retrieable errors by parsing message text.



Implement automatic retry logic at the tool level for technical errors only, passing business errors to the agent without retries.



Add a `check_refund_eligibility` tool that must be called before `process_refund`.



Return structured error responses with `retrieable` false for business errors and a customer friendly explanation.



Question 59 15 pts

Scenario: Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

A Claude Code session is running a complex refactor. Partway through, you need to pause and resume later. What should you use to continue the session?



Start a new session and manually repeat prior steps



Use `fork_session` to create a parallel continuation



Save the session output and use it as a new prompt



The `--resume` flag to continue the session with previous context intact



Question 60 15 pts

Scenario: Claude Code for Continuous Integration

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI review comments are repeating issues that were already discussed in previous reviews. What's the best fix?



Include prior review findings in the context so Claude Code recognizes already-discussed issues



Ask Claude Code to check previous comments before posting



Require developers to manually mark issues as resolved before re-review



Use a database to track commented issues and filter duplicates

Quiz saved at 10:08am

Submit Quiz